Advanced Operation System

Homework Report

Department of Computer Science and Engineering,

National Sun Yat-sen University, Kaohsiung.

ID : M153140005

Lecturer : 王冠智

Submission : 2026.10.22

Demonstration : 2026.11.03

Outline

- Directory
  - Homework Requirement
  - Reference String Implement
    - Random
    - Locality
    - Custom (Introduction)
  - Algorithm Implement
    - FIFO
    - MFU
    - ESC
    - Custom (Introduction)
  - Results

Homework Requirement

Homework

- 作業要求

1. 實作FIFO、MFU、ESC演算法並比較其效能
2. 設計一套演算法並在其中一項Benchmark中勝出
3. 繪製圖表並討論演算法行為

Implement

Implement

- Random Reference

reference > random_reference.py > ...

```python
import random

def generate_random_reference(num_references=150000, max_page=1500):

    random_reference = []

    for i in range(num_references):
        page = random.randint(1, max_page)
        random_reference.append(page)

    return random_reference
```

Implement

- Locality Reference

reference > locality_reference.py > ...

```python
import random

def generate_locality_reference(num_references=150000, max_page=1500):

    locality_reference = []
    procedure_lengths = []
    procedure_ranges = []

    # Procedure length: 1/200 to 1/100 of total references
    min_length = num_references // 200
    max_length = num_references // 100

    while len(locality_reference) < num_references:

        # Generate random procedure length
        procedure_length = random.randint(min_length, max_length)

        # Select 20 consecutive pages
        start_page = random.randint(1, max_page - 19)

        # Limit the total number of references
        remaining = num_references - len(locality_reference)
        procedure_length = min(procedure_length, remaining)

        # Generate page references
        for i in range(procedure_length):
            page = random.randint(start_page, start_page + 19)
            locality_reference.append(page)

        procedure_lengths.append(procedure_length)
        procedure_ranges.append((start_page, start_page + 19))

    return locality_reference, procedure_lengths, procedure_ranges
```

Implement

- Custom Reference

reference > custom_reference.py > ...

```python
import random

def generate_custom_reference(
    num_references=150000,
    max_page=1500,
    hot_pages=20,
    hot_probability=0.8,
    seed=None
):
    """
    Generate a Hotspot + Noise Reference String.
    Hotspot:
        Frequently access pages 1 to hot_pages.
    Noise:
        Occasionally access pages hot_pages+1 to max_page.
    Returns:
        List of page numbers.
    """
    # Validate parameters
    if num_references <= 0:
        raise ValueError("num_references must be positive")
    if not 1 <= hot_pages < max_page:
        raise ValueError("hot_pages must be between 1 and max_page-1")
    if not 0 <= hot_probability <= 1:
        raise ValueError("hot_probability must be between 0 and 1")
    # Use an independent random generator when seed is given
    rng = random.Random(seed) if seed is not None else random
    reference_string = []
    for _ in range(num_references):
        # Hotspot access
        if rng.random() < hot_probability:
            page = rng.randint(1, hot_pages)
        # Noise access
        else:
            page = rng.randint(hot_pages + 1, max_page)
        reference_string.append(page)
    return reference_string
```

Implement

- Custom Reference

1. 設計方式
   - 總存取次數： 150,000 次 Page References (作業要求)
   - Page 範圍： 1 ~ 1500 (作業要求)
   - 80% Hotspot Access： 集中存取 Page 1 ~ 20
   - 20% Random Access： 隨機存取 Page 21 ~ 1500
   - Read / Write Ratio： 約 70% Read、30% Write (所有Reference統一規格)
2. 設計目的
   - 模擬少數熱門頁面頻繁使用，以及偶發非熱門頁面存取
   - 評估演算法是否有效保留 Hot Pages，減少非必要 Page Fault

Implement

- FIFO

algorithms > fifo.py > ...

```python
from collections import deque

def fifo(reference_string, num_frames):
    if num_frames <= 0:
        raise ValueError("Number of frames must be positive")
    # Dictionary: page -> dirty bit
    memory = {}
    # FIFO replacement order
    queue = deque()
    page_faults = 0
    interrupts = 0
    disk_writes = 0
    for page, operation in reference_string:
        if operation not in ("R", "W"):
            raise ValueError("Operation must be R or W")
        # Page Hit
        if page in memory:
            # Writing makes the page dirty
            if operation == "W":
                memory[page] = 1
            continue
        # Page Fault
        page_faults += 1
        interrupts += 1
        # Memory is full
        if len(memory) >= num_frames:
            oldest_page = queue.popleft()
            # Write dirty page back to disk
            if memory[oldest_page] == 1:
                disk_writes += 1
                interrupts += 1
            del memory[oldest_page]
        # Load new page
        dirty_bit = 1 if operation == "W" else 0
        memory[page] = dirty_bit
        queue.append(page)
    return page_faults, interrupts, disk_writes
```

Implement

- MFU

algorithms > mfu.py > mfu

```python
def mfu(reference_string, num_frames):
    if num_frames <= 0:
        raise ValueError("Number of frames must be positive")
    # Pages currently in memory: page -> dirty bit
    memory = {}
    # Frequency counter for resident pages
    frequency = {}
    # Arrival order for tie-breaking
    arrival_order = {}
    arrival_time = 0
    page_faults = 0
    interrupts = 0
    disk_writes = 0
    for page, operation in reference_string:
        if operation not in ("R", "W"):
            raise ValueError("Operation must be R or W")
        # Page Hit
        if page in memory:
            frequency[page] += 1
            if operation == "W":
                memory[page] = 1
            continue
```

algorithms > mfu.py > mfu

```python
def mfu(reference_string, num_frames):
            # Page Fault
            page_faults += 1
            interrupts += 1
            # Memory is full
            if len(memory) >= num_frames:
                # Select page with highest frequency
                # Tie-breaker: oldest resident page
                victim = max(memory,key=lambda p: (frequency[p],-arrival_order[p]))
                # Write back dirty victim
                if memory[victim] == 1:
                    disk_writes += 1
                    interrupts += 1
                del memory[victim]
                del frequency[victim]
                del arrival_order[victim]
            # Load new page
            memory[page] = 1 if operation == "W" else 0
            frequency[page] = 1
            arrival_order[page] = arrival_time
            arrival_time += 1
    return page_faults, interrupts, disk_writes
```

Implement

- ESC

algorithms > enhanced_second_chance.py > enhanced_second_chance

```python
def enhanced_second_chance(reference_string, num_frames):
    if num_frames <= 0:
        raise ValueError("Number of frames must be positive")
    memory = []
    pointer = 0
    page_faults = 0
    interrupts = 0
    disk_writes = 0
    for page, operation in reference_string:
        if operation not in ("R", "W"):
            raise ValueError("Operation must be R or W")
        # Check Page Hit
        hit = False
        for frame in memory:
            if frame[0] == page:
                frame[1] = 1
                if operation == "W":
                    frame[2] = 1
                hit = True
                break
        if hit:
            continue
        # Page Fault
        page_faults += 1
        interrupts += 1
        dirty = 1 if operation == "W" else 0
        # Empty frame available
        if len(memory) < num_frames:
            memory.append([page, 1, dirty])
            continue
        # Enhanced Second-Chance victim selection
        while True:
            victim_index = None
            # Pass 1: Search for (R=0, M=0)
            # Do not modify R bits
            for _ in range(num_frames):
                frame = memory[pointer]
                if frame[1] == 0 and frame[2] == 0:
                    victim_index = pointer
                    break
                pointer = (pointer + 1) % num_frames
            if victim_index is not None:
                break
            # Pass 2: Search for (R=0, M=1)
            # Clear R bits while scanning
            for _ in range(num_frames):
                frame = memory[pointer]
                if frame[1] == 0 and frame[2] == 1:
                    victim_index = pointer
                    break
                frame[1] = 0
                pointer = (pointer + 1) % num_frames
            if victim_index is not None:
                break
        # Write back dirty page
        if memory[victim_index][2] == 1:
            disk_writes += 1
            interrupts += 1
        # Replace victim
        memory[victim_index] = [page, 1, dirty]
        # Advance clock pointer
        pointer = (victim_index + 1) % num_frames
    return page_faults, interrupts, disk_writes
```

Implement

- Custom Algorithm

algorithms > custom_algorithm_v2.py > ...

```python
from collections import deque, Counter
def custom_algorithm_v2(reference_string, num_frames):
    """
    DACR: Dirty-Aware Cost-adaptive Replacement
    Replacement decision considers:
    1. Recent access frequency
    2. Recency of page access
    3. Dirty bit
    4. Recent disk write pressure
    """
    if num_frames <= 0:
        raise ValueError("num_frames must be positive")
    # Parameters
    window_size = 2 * num_frames
    ## Recent eviction outcomes used to estimate
    ## disk write pressure
    pressure_window = max(20, 4 * num_frames)
    ## Maximum allowable difference in reuse score
    ## between a candidate and the lowest-score page
    candidate_margin = 0.25
    # Data structures
    ## Resident pages: page -> dirty bit
    memory = {}
    ## Last access time of each resident page
    last_used = {}
    ## Recent reference history
    history = deque()
    frequency = Counter()
    ## Recent eviction outcomes:
    ## 1 = dirty page evicted
    ## 0 = clean page evicted
    eviction_history = deque(maxlen=pressure_window)
    # Statistics
    page_faults = 0
    interrupts = 0
    disk_writes = 0
    replacement_count = 0
    changed_victim_count = 0
    clean_evictions = 0
    dirty_evictions = 0
    # Process reference string
    for step, (page, operation) in enumerate(reference_string, start=1):
        if operation not in ("R", "W"):
            raise ValueError("Operation must be R or W")
        ## Step 1: Update reference history
        history.append(page)
        frequency[page] += 1
        if len(history) > window_size:
            old_page = history.popleft()
            frequency[old_page] -= 1
            if frequency[old_page] == 0:
                del frequency[old_page]
        ## Step 2: Page hit
        if page in memory:
            last_used[page] = step
            if operation == "W":
                memory[page] = 1
            continue
        ## Step 3: Page fault
        page_faults += 1
        interrupts += 1
```

Implement

- Custom Algorithm

```
        ## Step 4: Replacement
        if len(memory) >= num_frames:
            replacement_count += 1
            resident_pages = list(memory.keys())
            ### Highest resident frequency for
            ### normalization
            max_frequency = max(frequency.get(p, 0) for p in resident_pages)
            max_frequency = max(1, max_frequency)
            ### Calculate page reuse scores
            reuse_scores = {}
            for p in resident_pages:
                #### Frequency score: 0 ~ 1
                freq_score = frequency.get(p, 0) / max_frequency
                #### Recency score
                age = step - last_used[p]
                recency_score = 1.0 / (1.0 + age / num_frames)
                #### Combined reuse score
                #### Prioritize frequency over recency
                reuse_scores[p] = 0.65 * freq_score + 0.35 * recency_score
            ### Baseline victim: lowest predicted reuse value
            normal_victim = min(resident_pages,key=lambda p: (reuse_scores[p],last_used[p]))
            minimum_score = reuse_scores[normal_victim]
            ### Candidate protection
            #### Only pages with sufficiently low
            #### reuse scores are eligible
            candidates = [
                p for p in resident_pages
                if reuse_scores[p] <= minimum_score + candidate_margin
            ]
```

```
                ### Estimate disk write pressure
                if eviction_history:
                    write_pressure = sum(eviction_history) / len(eviction_history)
                else:
                    write_pressure = 0.0
                #### Higher write pressure means stronger penalty for dirty pages
                dirty_penalty = 0.15 + 0.55 * write_pressure
                ### Dirty-aware victim selection
                victim = min(candidates,key=lambda p: (reuse_scores[p] + dirty_penalty * memory[p], last_used[p]))
                if victim != normal_victim:
                    changed_victim_count += 1
                ### Evict selected page
                was_dirty = memory[victim]
                if was_dirty == 1:
                    disk_writes += 1
                    interrupts += 1
                    dirty_evictions += 1
                else:
                    clean_evictions += 1
                ### Update recent write pressure
                eviction_history.append(was_dirty)
                del memory[victim]
                del last_used[victim]
            ## Step 5: Load the requested page
            memory[page] = 1 if operation == "W" else 0
            last_used[page] = step
    return page_faults, interrupts, disk_writes
```

Implement

- Custom Algorithm

1. 設計動機
   - 針對Hot Page模式，以LFU(保留常用Page)為基礎來設計
   - 欲減少Disk Write次數，故套用Dirty Penalty來賦予演算法對Disk Write的敏感度
2. 設計目標
   - 保留常用Pages
   - 避免Dirty Bit頻繁寫入記憶體
   - 計算淘汰成本(Cost)以決定淘汰目標

Implement

- Custom Algorithm

3. 演算法運作流程
   a. 用Sliding Window紀錄近期Page出現頻率
   b. 考慮近期出現頻率跟近期使用率(賦予權重)，評估保留價值
   c. 篩選保留價值低的Pages作為候選人(Candidate)
   d. 依Dirty Eviction比例調整懲罰係數(Dirty Penalty)
   e. 從Candidate中選取Cost最低的進行淘汰

Implement

- Custom Algorithm

3. 演算法運作流程
   a. 用Sliding Window紀錄近期Page出現頻率
      1) ***W*** (Window Size) ***= 2 * num_frame*** (Frame數量)
      2) Deque存取近W次的Page Reference (僅存取部分歷史)
      3) 統計Window中，各Page出現的次數
      4) Window滿了就刪掉最舊的紀錄
   b. 考慮近期出現頻率跟近期使用率(賦予權重)，評估保留價值
   c. 篩選保留價值低的Pages作為候選人(Candidate)
   d. 依Dirty Eviction比例調整懲罰係數(Dirty Penalty)
   e. 從Candidate中選取Cost最低的進行淘汰

Implement

- Custom Algorithm

3. 演算法運作流程
   a. 用Sliding Window紀錄近期Page出現頻率
   b. 考慮近期出現頻率跟近期使用率(賦予權重)，評估保留價值
      1) ***F(p) = Frequency(p) / MaxFrequency***
      2) ***R(p) = (1 + Age(p)/num_frame)$^{-1}$***
      3) ***ReuseScore(p) = 0.65F(p) + 0.35R(p)*** (分數越高則越有保留價值)
   c. 篩選保留價值低的Pages作為候選人(Candidate)
   d. 依Dirty Eviction比例調整懲罰係數(Dirty Penalty)
   e. 從Candidate中選取Cost最低的進行淘汰

Implement

- Custom Algorithm

3. 演算法運作流程
   a. 用Sliding Window紀錄近期Page出現頻率
   b. 考慮近期出現頻率跟近期使用率(賦予權重)，評估保留價值
   c. 篩選保留價值低的Pages作為候選人(Candidate)
      1) 找 ReuseScore(p) 中最低的 (S_{min})
      2) ***Threshold = S_{min} + 0.25*** (將ReuseScore(p)在這個值以下Page的列入CandidateSet)
   d. 依Dirty Eviction比例調整懲罰係數(Dirty Penalty)
   e. 從Candidate中選取Cost最低的進行淘汰

Implement

- Custom Algorithm

3. 演算法運作流程
   a. 用Sliding Window紀錄近期Page出現頻率
   b. 考慮近期出現頻率跟近期使用率(賦予權重)，評估保留價值
   c. 篩選保留價值低的Pages作為候選人(Candidate)
   d. 依Dirty Eviction比例調整懲罰係數(Dirty Penalty)
      1) ***P = DirtyEviction / HistoryEviction*** (近期淘汰事件中產生Disk Write次數)
      2) ***λ = 0.15 + 0.55P*** (Dirty Penalty)
   e. 從Candidate中選取Cost最低的進行淘汰

Implement

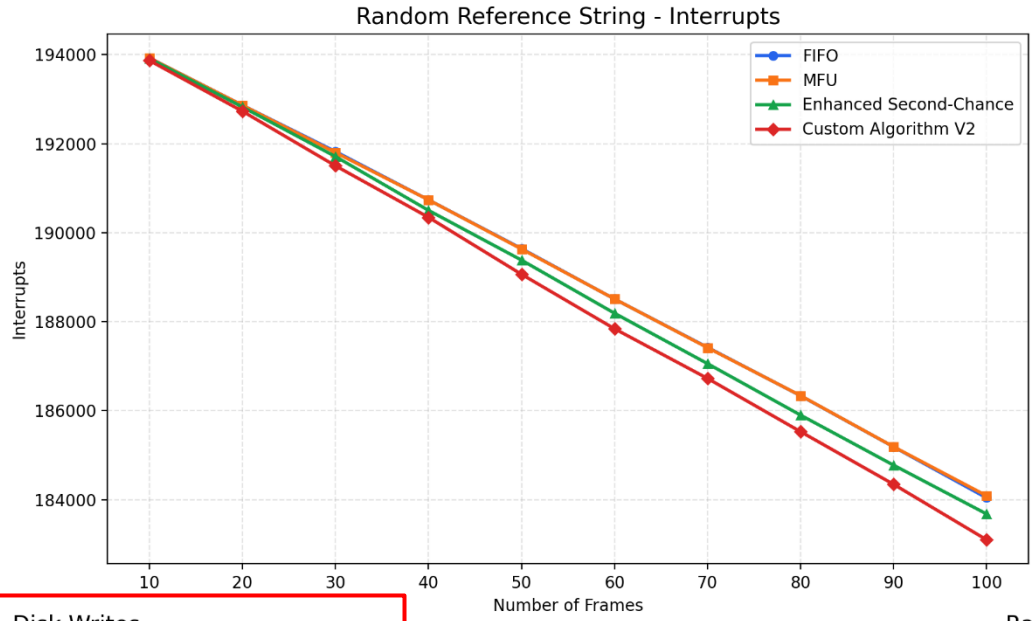
- Custom Algorithm

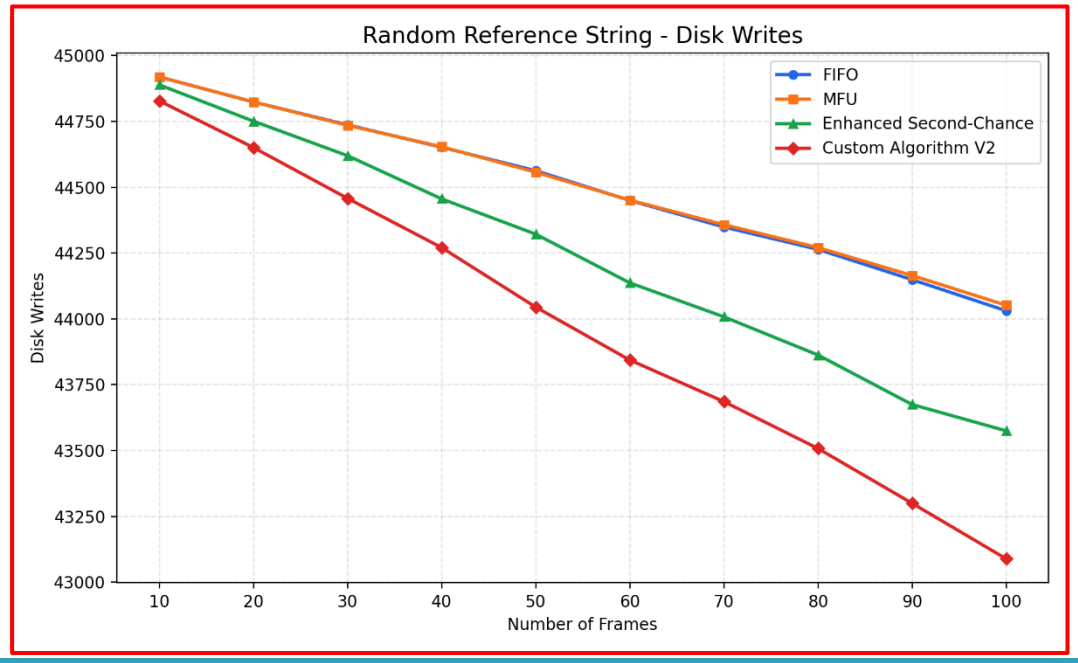
3. 演算法運作流程
   a. 用Sliding Window紀錄近期Page出現頻率
   b. 考慮近期出現頻率跟近期使用率(賦予權重)，評估保留價值
   c. 篩選保留價值低的Pages作為候選人(Candidate)
   d. 依Dirty Eviction比例調整懲罰係數(Dirty Penalty)
   e. 從Candidate中選取Cost最低的進行淘汰
      1) ***Cost(p) = ReuseScore(p) + λD(p)*** (淘汰最低Cost的Page)

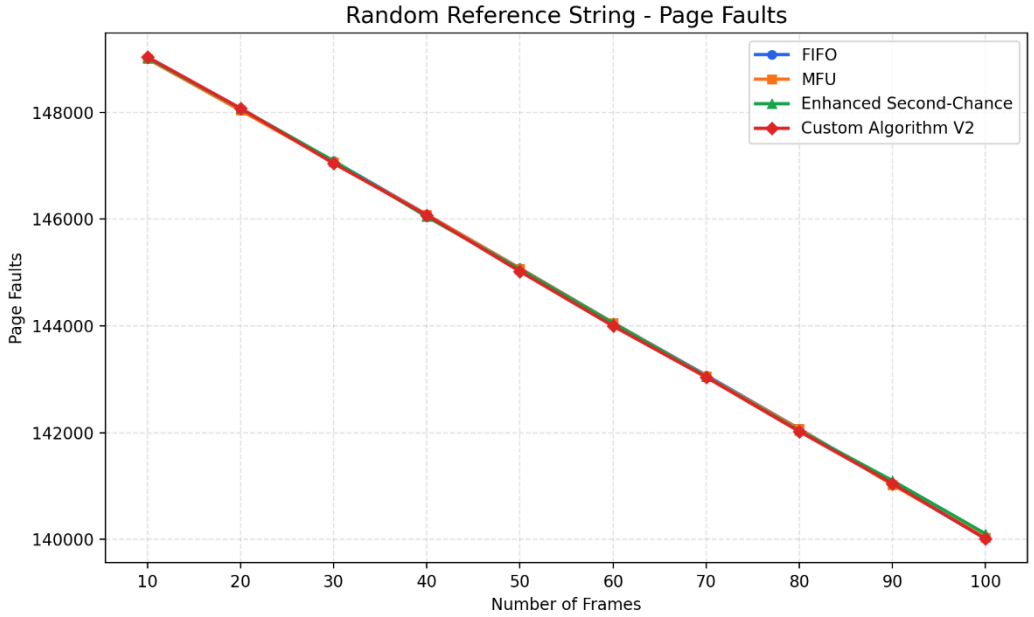
Results

Result

- Random Page Reference

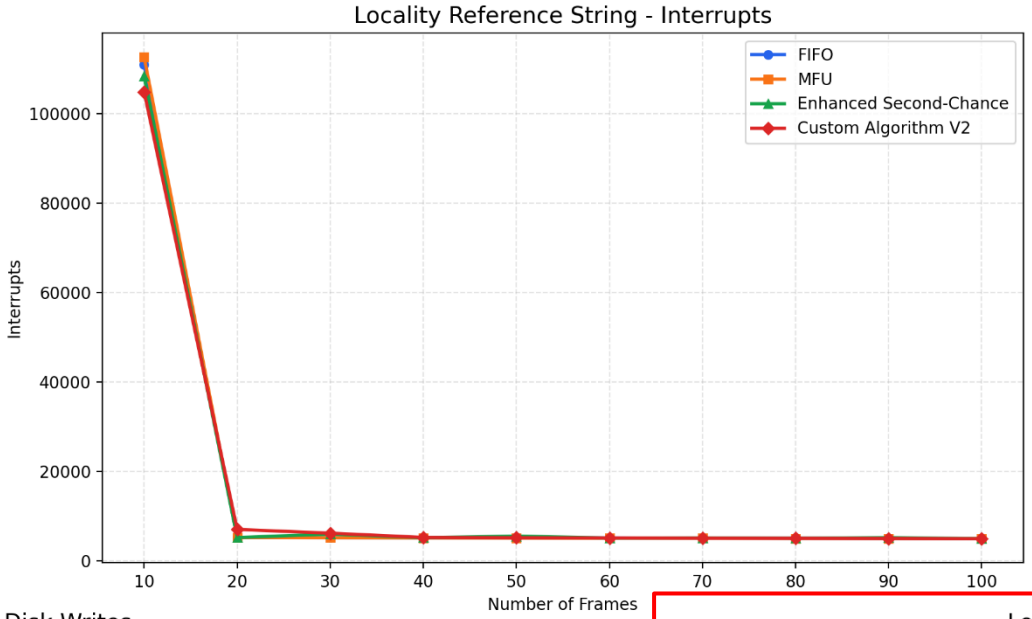


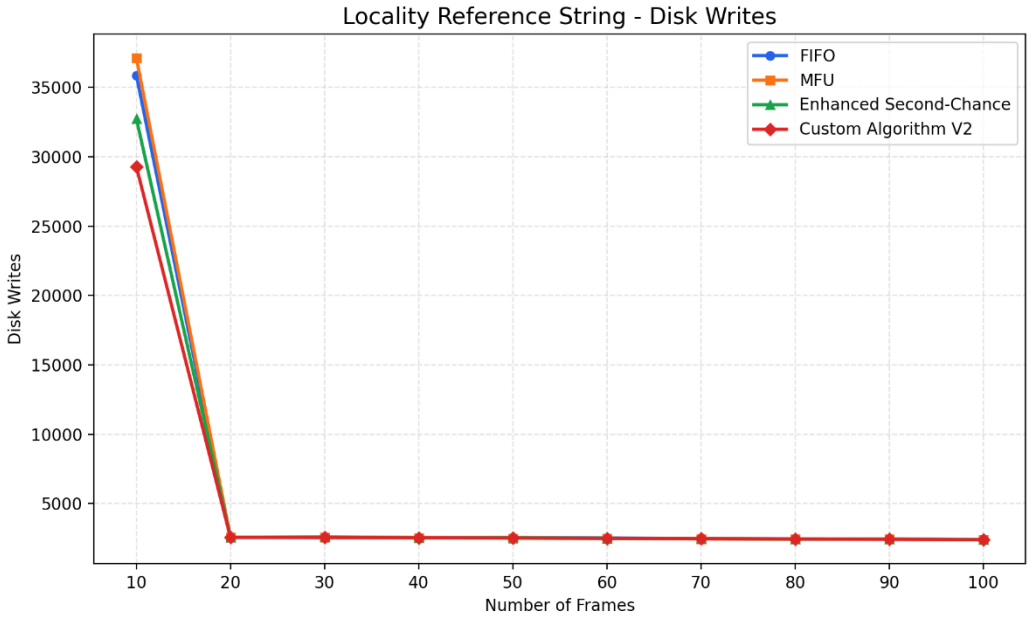


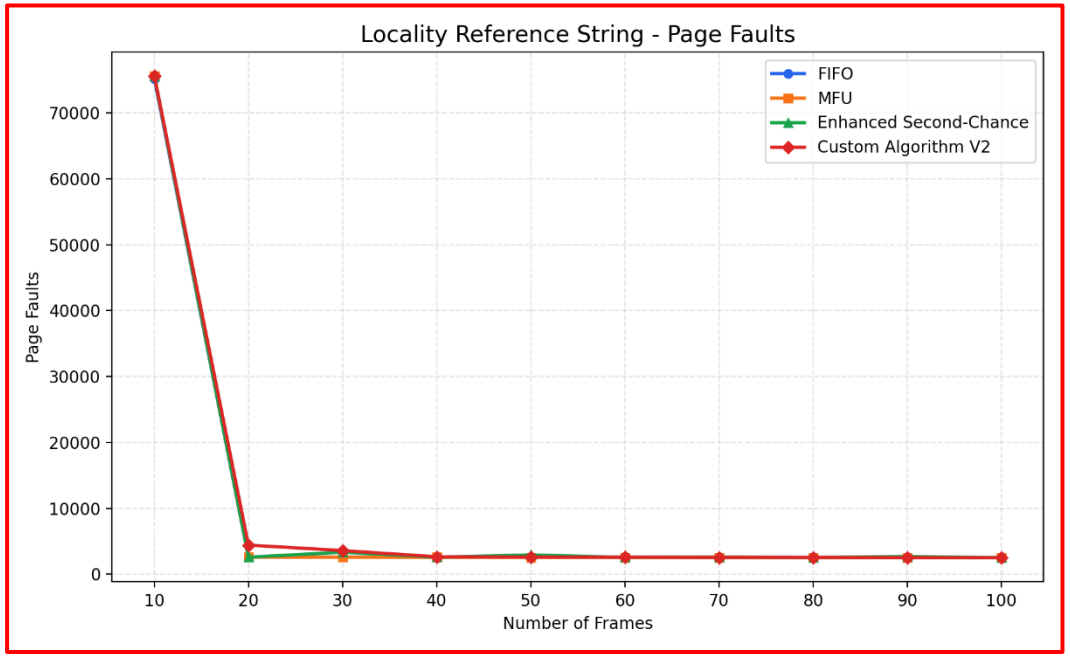


Result

- Locality Page Reference







Result

- Custom Page Reference

